



Estimating energy consumption of neural networks with joint Structure–Device encoding

Chaopeng Guo, Shiyu Wang, Ruolan Xie, Jie Song^{*}

Software College, Northeastern University, No. 195, Chuangxin Road, Hunnan District, Shenyang, 110169, Liaoning, China

ARTICLE INFO

Keywords:

Energy estimation
Neural networks
Structure encoding
Device encoding

ABSTRACT

The surge in IoT devices has led to an increase in energy consumption, necessitating the optimization of neural networks deployed on these energy-constrained devices to reduce power usage. Although various techniques, such as pruning and quantization, can reduce the size and computational requirements of neural networks, the resulting energy savings still need to be verified through resource-intensive inference processes, which require cumbersome adjustments to measurement devices and neural network deployment. To address these challenges, we propose SDEnergy, a novel approach that combines Structure–Device encoding to quickly and accurately predict the Energy consumption of neural networks across various devices. SDEnergy utilizes graph neural networks to extract structural features of neural networks and employs fully connected networks to extract device features, using their fusion for energy consumption prediction. Experimental validation demonstrates that SDEnergy has established state-of-the-art results on our dataset based on NAS-Bench-101 and various IoT device parameter scenarios, with a mean absolute percentage error of 5.35%.

1. Introduction

Internet of Things (IoT) devices are limited in terms of power supply and energy consumption due to factors such as cost considerations, power constraints, and environmental impact. They are typically small, portable, and battery-powered. However, numerous neural networks with high energy consumption are deployed on these devices for various deep learning tasks. This energy consumption accounts for up to 90% of the machine learning costs at deployment time [1–3]. Therefore, improving the energy efficiency of neural network inference is critical.

Some studies have optimized energy consumption from the perspective of algorithms or the deployed devices. For instance, Yin et al. [4] utilized the Grey Wolf Optimization Algorithm to enhance the performance of algorithms in practical optimization problems, particularly in terms of energy conservation in production line scheduling solutions. Additionally, Yang et al. [5] started from the device and circuit level, capable of providing higher power at a lower cost. The starting points of energy efficiency optimization across different fields can be applied to the one of neural network inference, where improvements in algorithms or optimizations in hardware can achieve a certain level of energy efficiency enhancement.

From the perspective of algorithms, lightweight approaches to neural networks, such as pruning [6,7] and quantization [8,9], can significantly reduce the number of parameters and energy consumption

of neural networks, making them more energy-efficient. However, measuring energy consumption during the optimization process of lightweight neural networks can be challenging due to the multitude of temporary structures and the diversity of target devices. Performing network inference for each temporary structure would result in an unacceptable execution time. Furthermore, the optimization result cannot be reused for different target devices, increasing development costs. Therefore, a fast and accurate method for predicting energy consumption is essential.

In recent years, machine learning and deep learning techniques have been applied to predict energy consumption for neural networks. For instance, NeuralPower [10] employed polynomial regression models to estimate power and time separately, while ChamNet [11] used Gaussian process regression. PreVlous [12] employed linear regression models for each layer and fitted a coefficient to bridge the gap between layer-by-layer estimates and overall energy consumption. SyNERGY [13] computed the entire neural network's floating-point operations (FLOPs) and adopted a linear regression model. Justus et al. [14] used fully connected neural networks to extract features from layers and summed each layer's predicted energy consumption to obtain the total network energy consumption. EALI [15] and Getzner et al. [16] provided insights into the energy behavior of various layer types and built layer-wise energy prediction models.

^{*} Corresponding author.

E-mail address: songjie@mail.neu.edu.cn (J. Song).

However, there are some limitations in the existing methods: (1) Most prior studies have overlooked the comprehensive structural characteristics of the entire network, focusing mainly on FLOPs or layer-wise energy consumption as predictive features. (2) The energy consumption varies significantly across different target devices due to varying computational capabilities. Unfortunately, these device-specific factors are often disregarded, resulting in poor transferability. This is because energy consumption is not only related to the structure of the neural network but also to the device on which it is deployed [17,18]. (3) The energy prediction problem is typically modeled as a supervised learning task, but model performance may suffer when training data is limited.

To overcome the above challenges, we introduce SDEnergy, a novel approach for predicting the energy consumption of various neural networks with joint Structure-Device Encoding across different devices. SDEnergy integrates structural characteristics of neural networks with specific device features to provide a comprehensive energy consumption model.

We employ the variational autoencoder (VAE) schema [19] as the foundational architecture for the energy predictor, leveraging encoders to extract pertinent features and decoders to predict energy values. Firstly, we incorporate a Graph Neural Network (GNN) into the encoders to extract the structural features of the neural network. GNN variants such as Graph Convolutional Networks (GCN) [20], Graph Attention Networks (GAT) [21], and Graph Isomorphism Networks (GIN) [22], have been widely used to learn the structural features of graph data. In the IoT domain, there are also some applications utilizing graph neural networks [23,24], and there have been some breakthroughs in foundational methods as well [25,26]. In this study, we treat the structures of neural networks as graph representations. We propose a Bi-directional Graph Neural Network Model (BiGNN) specifically designed to extract the inherent structural features of neural network models.

Beyond these neural network features, we also account for device-specific factors in the encoder design to handle the influence of varying target devices. Device-specific factors, such as memory size, CPU frequency, storage bandwidth, and latency, are incorporated into a branch of the model and managed by our encoder. These key indicators, used to gauge different devices, are processed through a Fully Connected Network (FCN) to yield vectors of feature representation. These extracted structural features and device features are subsequently fed into a decoder to predict the energy consumption on the specified target device.

To address the limitations of training data, we adopt semi-supervised training methods such as BERT [27] and MAE [28]. The training task is split into two sequential phases: pre-training and fine-tuning. During the pre-training phase, our exclusive focus is on the neural network's structural encoder. We train this encoder unsupervisedly by masking portions of the neural network's structure, subsequently aiming to reconstruct them. Subsequently, in the fine-tuning phase, both the device encoder and the energy prediction decoder are concurrently trained to refine the model's predictive capabilities.

The primary contributions of our study are as follows:

- A predictive model for energy consumption is developed, which integrates a Graph Neural Network (GNN) encoder to extract structural features of the network and a device encoder to incorporate device-specific characteristics. This model leverages both device features and network structural attributes to achieve a state-of-the-art performance in energy consumption forecasting.
- A novel bi-directional Graph Neural Network is introduced, termed BiGNN, which accounts for information flow in both the parent-to-child and child-to-parent directions. Incorporating our unsupervised training for BiGNN, it has achieved better results in graph structural representation learning for energy consumption prediction.
- A two-stage training methodology is implemented, which integrates the unsupervised training of Graph Auto Encoders with supervised training for energy consumption prediction based on graph structures and device encoding fusion. The first stage of unsupervised training enhances the representation learning of the graph structures derived from neural networks, thereby improving the efficacy of energy consumption prediction.
- Extensive experimental validation is conducted to substantiate the efficacy of the proposed model and training methodology in accurately predicting the energy consumption of neural networks across a spectrum of devices.

The remainder of this paper is organized as follows. Section 2 shows the works related to ours. Section 3 provides an overview of our work, and Section 4 shows the details mentioned in Section 3. Section 5 shows the processes of pre-training and fine-tuning. Section 6 includes the details and results of the experiment. Finally, Section 7 concludes the paper and proposes our future directions.

2. Related work

By combining structure features of neural networks and parameter features of target devices, we propose SDEnergy to predict energy consumption, improving accuracy and transferability compared to previous works. This section summarizes previous energy consumption prediction works and introduces related studies, including energy prediction approaches and graph neural networks.

2.1. Energy prediction approach

The current approaches for predicting energy consumption during neural network inference can be grouped into three categories: measurement-based approaches, analysis-based approaches, and estimation-based approaches.

The measurement-based approaches directly measure the energy consumption of the target device by a power meter or other devices [29–31]. The advantage is that the approaches are simple and provide accurate energy estimations that are not specific to particular neural network structures or operating devices. However, they have several limitations. Firstly, the granularity of the measurement is relatively large, as they can only measure the overall energy consumption of the neural network and cannot measure the energy consumption of individual hidden layers. Therefore, the methods cannot provide fine-grained optimization guidance for energy consumption optimization methods, such as identifying high-energy-consuming hidden layers, which is not conducive to subsequent analysis and further optimization. Secondly, measurement-based approaches typically require multiple measurements to obtain accurate energy consumption. However, existing energy optimization methods are iterative and rely on real-time energy estimation results, which presents a performance bottleneck. Thirdly, the methods require the use of measurement devices and cannot be widely used for large-scale energy estimation, such as energy estimation in distributed environments. Therefore, in current research, they are commonly used to collect raw, coarse-grained energy consumption data.

The analysis-based approaches typically consider the factors related to energy consumption at the inference time, such as computation, data access amount, and communication amount. Han et al. [9] considered the computation, communication, and back-propagation energy consumption in their research. They calculated the energy consumption of inference by counting Multiply-Accumulate Operations (MAC) and activation operations. Qi et al. [32] and Yang et al. [17] considered computation and data access energy consumption, where the former is related to the number of neurons and weights, and the latter is related to the number of data storage units and access times. The analysis-based approaches do not require the actual operation of the neural

network. The approaches can obtain the network energy consumption in different scenarios by calculating the energy consumption of all operations. They have a low time cost but require analysis and calculation of the structure and parameters of neural networks. Therefore, the accuracy is limited by factors such as device feature extraction, parameter estimation, and model assumptions.

In recent years, many studies have adopted methods such as linear regression and deep learning to predict the energy consumption of neural networks, defined as the estimation-based approaches. The approaches no longer analyze factors such as Floating Point Operations (FLOPs) and data storage units. Still, They consider the impact of several factors related to energy consumption, such as the parameters of different layers of neural networks and the graph structure formed by their connections. NeuralPower [10] established a polynomial regression model based on the types of different layers in neural networks, which can predict the inference time and corresponding power consumption of each layer. PreVIOUS [12] fitted a coefficient to fill the gap between the per-layer prediction and the overall energy consumption and then adjusted the energy prediction accordingly. SyNERGY [13] constructed a regression model based on the total MAC count of the network. However, they divided the entire neural network into several modules and ignored the whole structure. In SDEnergy, the structure feature was taken into account. In addition, Justus et al. [14] applied fully connected neural networks to extract layers' features and sum each layer's predicted energy consumption as the entire network's consumption. EALI [15] proposed an energy-aware layer-level adaptive scheduling framework that is comprised of the power prediction model, a layer combination strategy, and an energy-aware layer-level scheduler. Getzner et al. [16] proposed an energy prediction baseline, offering insights into the energy behavior of various layer types. In these works, the device factors were completely ignored, which led to poor transferability.

The estimation-based approaches are relatively simple, since they can predict the future energy consumption of neural networks by learning from historical data. These approaches also take into account the influence of various hierarchical and structural parameters of the network on energy consumption, resulting in more accurate prediction results compared to the analytical method. However, the estimation approaches require learning different structures, making the process of data collection and model construction more complex. Furthermore, existing research does not consider the dynamic changes in network energy consumption under different device environments. Based on the aforementioned observations, this work proposes SDEnergy, which adopts an estimation-based approach to predict the energy consumption of a neural network. In contrast to previous works, SDEnergy not only considers the structural features of neural networks but also takes into account the impact of different target devices.

2.2. Graph neural networks

We utilize graph neural networks to extract network structural features. In this section, we provide a brief introduction to graph neural networks. Graph neural networks are a class of neural network models based on graph-structured data, which can learn feature representations of nodes and edges and perform tasks such as node classification, edge prediction, and graph classification. Unlike traditional neural networks, graph neural networks consider the topological structure information between nodes, which allows them to propagate and interact between nodes in the graph.

For example, GAT [21] proposed a method that uses attention mechanisms to weigh the neighboring node features. The weights of the neighboring node features depend solely on the node features and are independent of the graph structure. GCN [20] applied the adjacency matrix to perform convolution operations on the input feature matrix. It aggregates the features of neighboring nodes to the central node and then processes them through an activation function to obtain a new

representation of the node features. This process can be stacked layer by layer, gradually expanding the scope of information propagation and aggregation, ultimately obtaining a global feature representation.

In the fields of the Internet of Things (IoT) and industrial applications, Bi et al. [23] focused on representation learning for incomplete industrial data, enhancing the accuracy of representation learning by introducing proximal regularization and incorporating non-negativity constraints in autoencoders to reduce reconstruction errors. Liu et al. [24] also improved the representation learning capabilities for large-scale undirected networks by introducing the Symmetry and Graph Bi-regularized Non-negative Matrix Factorization method in graph neural networks. A knowledge graph-based system provided a hierarchical integration of IoT data [25], capable of handling multi-source data from personalized products and assisting decision-making through graph embedding methods. mGNN [26] achieved sensitive learning for a minority of nodes using a weighted aggregation approach and enhanced the predictive performance for minority nodes with an algorithm activated by the Gumbel distribution.

Training of graph neural networks can be done with an approach of reconstruction-based unsupervised learning, using the local structure information of the graph data to reconstruct nodes or edges. It can learn the embedding representation of the graph data by optimizing the reconstruction loss. Classic methods for it include auto-encoders and graph embedding reconstruction. It can deal with the problem of lacking training data for supervised learning. Auto-encoders [33] are a commonly used unsupervised learning method that compresses input data into a low-dimensional representation and then reconstructs it back to the original space through a decoder. In graph data, the input of an auto-encoder is a subgraph, and the output is the embedding representation of it. This method can learn the embedding representation of nodes by minimizing the reconstruction loss while preserving the local structure information of the graph. In this paper, we apply the auto-encoder method and improve the training process with the GraphMAE-based [34] approach.

The method for energy consumption prediction in this paper, in simple terms, involves a neural network architecture and the device on which this network is deployed. By encoding the graph structure and device information, along with feature fusion techniques, energy consumption data is predicted in a regression format. That is to say, the prediction outcome corresponds to the energy consumption of a neural network performing a fixed number of inferences on a device. In terms of processing the graph structure, we consider not only the direction of data flow within the neural network but also the interactions between child and parent nodes. Therefore, a bi-directional Graph Neural Network (BiGNN) has been proposed for spatial structure. Although our target does not involve processing on the temporal dimension, such bi-directional relationships have been well applied on the temporal dimension based on LSTM or GRU [35,36], and these methods can also handle serialized data quite effectively. However, the neural network in this paper is graph-structured data, which is the reason why GNN is used to process it in this paper.

3. Overview

Previous works mostly utilized GNN to extract network structure information and employed supervised training methods based on network-energy correspondence. Some approaches first pre-trained a network structure encoder on a large dataset and then fine-tuned it to obtain an energy predictor. However, these methods lack device-specific information, resulting in poor transferability, i.e., the inability to predict network energy in different device scenarios.

Based on the observation of these limitations, SDEnergy applies fully connected networks to encode parameters of important factors in the devices and applies GNN for encoding neural network structural features. The training of the structural feature extractor is conducted using unsupervised training methods for pre-training. Subsequently,

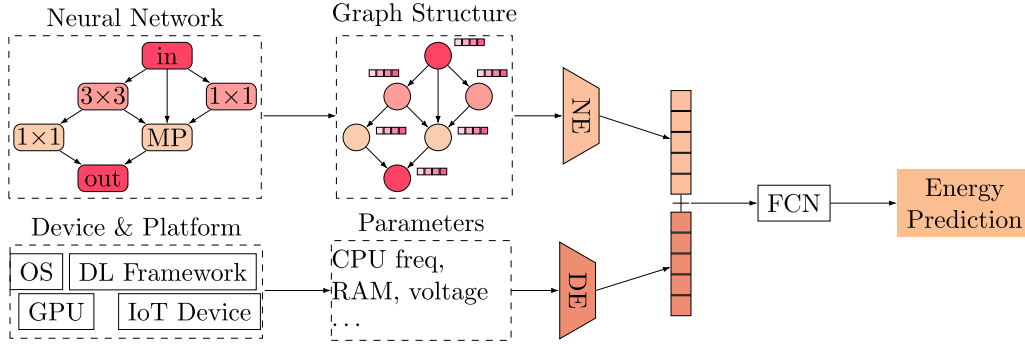


Fig. 1. Overview of SDEnergy. A neural network can be represented as a graph structure of a directed acyclic graph (DAG) with a feature vector on each node. A device, containing an operating system (OS), GPU, Deep Learning (DL) Framework, and so on, can be described by some hardware or software parameters, such as CPU frequency, and random access memory. The graph structure can be fed into the Network Encoder (NE) and the description of the device can be handled by the Device Encoder (DE) to get two vectors of feature representation. The two vectors are concatenated as the input of a fully connected network (FCN). The FCN works as the predictor to get the energy prediction result.

the structural features and device parameter features are combined, a Predictor is added, and fine-tuning is performed using actual measured energy data collected. The overview of SDEnergy is shown in Fig. 1.

The graph-level representation of the graph structure is obtained using a Network Encoder, which extracts feature representations of the network structure. The graph structure consists of nodes G_v and edges G_e . The Network Encoder takes them as inputs and generates a feature representation h that represents the entire graph structure. The process of graph encoding by Network Encoder is shown as Eq. (1), where x_v is the feature embedding of v , generated by an embedding layer.

$$h = \text{network_encoder}(\{x_v | v \in G_v\}, G_e) \quad (1)$$

Regarding the representation of graph structures, the features of each node are stored in a matrix with the shape (N_{node}, D_{feat}) , where N_{node} is the number of nodes and D_{feat} means the feature dimension. Meanwhile, the connections between two nodes are stored to indicate whether there is a connection between them, and the edges are saved in a matrix with the shape $(N_{edge}, 2)$, where N_{edge} is the number of edges. Details including the node masking method and the structure of the Network Encoder (NE) are in Section 4.1.

The encoding of device parameters is applying FCN to transform the parameters x that represent hardware information into a feature vector c . The device encoding process by Device Encoder is shown as Eq. (2), where x_{sth} depends on the hardware device parameters related to energy consumption.

$$c = \text{device_encoder}([x_{cpu}, x_{gpu}, x_{sys}, \dots]) \quad (2)$$

Before the energy consumption prediction process, the encoded results of the graph structure h and the device information c are combined in a concatenated way. By inputting these two feature vectors into the energy predictor, the predicted result of energy consumption e can be obtained, and the process of predictor is shown as Eq. (3) where concat means concatenating network encoding h and device encoding c .

$$e = \text{predictor}(\text{concat}(h, c)) \quad (3)$$

By employing this approach, SDEnergy integrates the neural network's graph structure information with the corresponding device parameters to predict energy consumption. It improves the predictive performance for network energy consumption and overcomes the limitations of traditional methods across different device scenarios.

Additionally, in the training process, an unsupervised method is used for pre-training, while fine-tuning is performed using measured energy consumption data collected. For the training of the Network Encoder, we initially generate a large amount of graph-structured data and randomly mask a portion of node feature information or edge information for each graph. During the pre-training phase, the objective of the Network Encoder is to reconstruct the masked node feature in the

graph. Regarding the training of the Device Encoder, it directly participates in the subsequent fine-tuning process as a supervised training. During the fine-tuning phase, the pre-training Network Encoder and Device Encoder, as well as the Predictor, are applied with the aim of enhancing the accuracy of energy consumption data prediction.

4. Method

This section presents the methodology of the proposed model, which mainly consists of a Network Encoder, Device Encoder, and Decoder. Additionally, the training process will be described, including the methods employed during the pre-training phase, such as graph masking and the selection of the Loss Function. The fine-tuning phase will cover the training approach and the objective function used.

4.1. Network Encoder

The network used as input in this paper can be obtained from a graph-structure dataset, which should contain a large number of network structures. Let $\mathbf{G}$ denote this dataset, and take a neural network $\mathcal{G} \in \mathbf{G}$ as an example. The network layers of $\mathcal{G}$ can be viewed as nodes in a graph structure, and the connections between the layers can be viewed as edges. These two aspects form two sets, $\mathbf{V}$ and $\mathbf{E}$, respectively. Thus, a graph $\mathcal{G}$ can be represented as $\mathcal{G} = (\mathbf{V}, \mathbf{E})$.

In our implementation, each node $v \in \mathbf{V}$ has a category cls_v indicating the class of a neural network layer. The category is embedded into a vector x_v as the feature representation of the node, which can be formulated as $x_v = \text{emb}(cls_v)$.

The set of edges is stored as an adjacency matrix $\mathbf{A}$, which is an upper triangular matrix of size $|\mathbf{V}| \times |\mathbf{V}|$. If there exists a connection between two nodes $v_i, v_j \in \mathbf{V}$, denoted as $e_{ij} \in \mathbf{E}$, then the corresponding entry in the matrix, $\mathbf{A}_{ij}$, is set to 1; otherwise, $\mathbf{A}_{ij}$ is set to 0.

At the training phase, we randomly select a subset of nodes $\bar{\mathbf{V}} \subset \mathbf{V}$, and mask the feature x_v of the nodes $v \in \bar{\mathbf{V}}$, while preserving the structural information. The nodes involved in this masking operation are denoted as $v_{[M]}$. Similarly, for a subset of edges $\bar{\mathbf{E}} \subset \mathbf{E}$, we mask the edges $e \in \bar{\mathbf{E}}$, denoted as $e_{[M]}$. We choose $\mathcal{G}_v = (\bar{\mathbf{V}}, \mathbf{E})$ and $\mathcal{G}_e = (\mathbf{V}, \bar{\mathbf{E}})$ as the input to two encoders during training process respectively. The Network Encoder structure and its training process are shown in Fig. 2.

$$h_v = f(\mathcal{G}_v; \theta) \quad (4)$$

$$h_e = f(\mathcal{G}_e; \theta) \quad (5)$$

These two encoders, one for learning node information and the other for learning edge information, are called Feature Encoder and Structure Encoder, shown as Eq. (4) and (5). They share the same structure and weight values and are represented as $f(\cdot; \theta)$. The graph-level representations learned by these encoders are denoted as h_v and h_e , which indicates the information of nodes and edges. The schema of

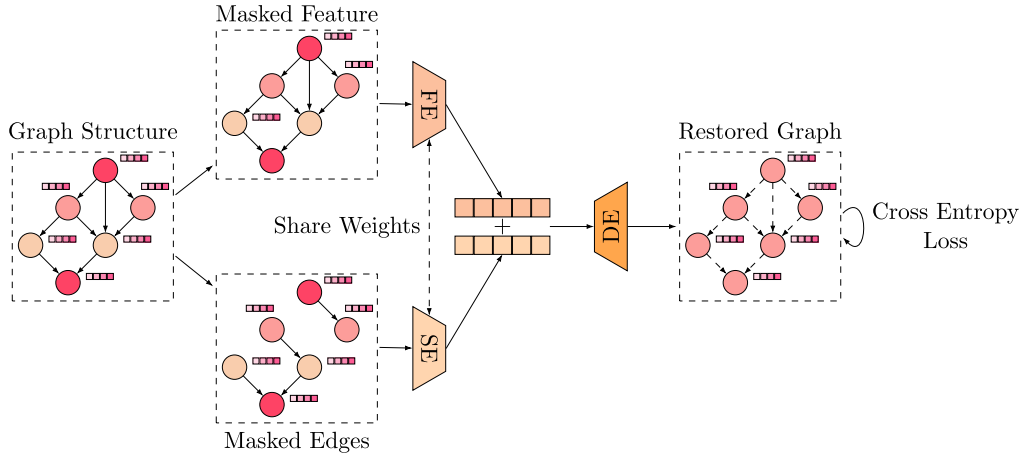


Fig. 2. Training process of Network Encoder and method of masking. A graph structure can be masked on the nodes or edges. We randomly select some nodes and mask the feature of them as the Masked-Feature Graph. As for the edges, some of them are masked the masked graph is the Masked-Edges Graph. FE and SE indicate the Feature Encoder and the Structure Encoder, and the two encoders share the same weights. The output features of them are added and fed into the Decoder (DE). The training objective of the Decoder is to restore the masked feature of selected nodes in the initial graph.

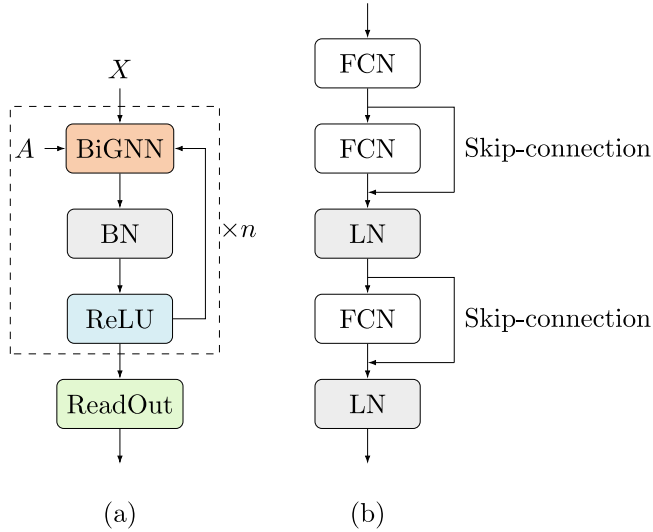


Fig. 3. (a) Combination mode of BiGNN in the Feature Encoder and (b) Structure of FEL.

the Feature Encoder (FE) is shown in Fig. 3(a), which is also the schema of the Structure Encoder (SE). In our implementation, they initially receive the feature representation of all nodes in a DAG, denoted as X , and the adjacency matrix A at each loop. BN represents the Batch Normalization process, ReLU is the activation function and the ReadOut layer is an average pooling function. The BiGNN, a bi-directional graph neural network, is our proposed method to deal with the DAG and will be shown in Section 4.3.

During the inference phase, the trained encoder takes the entire set of nodes V and the set of edges E , which constitute a graph structure of a neural network, as input. Consequently, the encoder produces the structural features of the network as a part of SDEnergy.

4.2. Decoder

In the Network Encoder, the schema of the decoder, denoted as DE in Fig. 2 is an FCN followed by a softmax layer. It receives the outputs h_v and h_e from the Network Encoder as inputs only during the pre-training phase. This decoder decodes the information h containing structural features from the encoders, shown as Eq. (6).

$$DE(h) = \text{Softmax}(\text{FCN}(h)) \quad (6)$$

The training method of the decoder is an unsupervised learning approach. We first randomly mask some node features and edges of the graph, and the objective of the decoder is to reconstruct the node category in the masked graph structure.

Based on the objective of training, the loss function involved needs to separately compute the recovery metric for node category, and the Cross-Entropy Loss (CELoss) is employed. As for each node, the loss between the real node category y and the output of the decoder $DE(h)$ is computed, shown as Eq. (7).

$$Loss_{NE} = \text{CELoss}(DE(h), y) \quad (7)$$

4.3. BiGNN

A neural network, as a directed graph, has a limitation on information flow during aggregation. In directed graphs, nodes can only gather information from their children, not their parents. This can lead to biases in node and graph embeddings if we treat the neural network as an undirected graph. To address this issue, we introduce BiGNN, a bi-directional Graph Neural Network, and its structure is shown in Fig. 4. BiGNN collects features from both parent and child nodes separately. In this section, an algorithm of extracting structural features based on BiGNN is provided.

Given a DAG $\mathcal{G}$, we have its feature representation of nodes $X = \{\text{emb}(cls_{v_i})\}, i = 1, 2, \dots, n$, where n denotes the number of nodes in $\mathcal{G}$. BiGNN first projects the input X into a desired hidden representation X_F using the feature extraction layer (FEL), which is shown in Fig. 3(b). In FEL, three FCN layers are stacked to improve the fitting ability of BiGNN and make it easier for the following aggregator to handle these features. Additionally, the skip-connection mechanism and the Layer Normalization are utilized between the FCN layers to avoid the gradient vanishing problem and accelerate the training process. Then, BiGNN takes the hidden representation feature, adjacency matrix A , and the transposed adjacency matrix A^T as inputs.

BiGNN consists of two-stream aggregators: the forward aggregator handles A , and the reverse aggregator handles A^T . These two aggregators capture non-overlapping structural features. To separate these directions, we employ a new weight matrix and denote the hidden presentations as O and O_r for the forward and reverse aggregator, respectively, with dimensions H and H_r .

In the aggregator, the graph attention layer (GAT) receives the graph $\mathcal{G} = (V, E)$ as input. For each node $v_i \in V$, we have its feature representation $x_i \in X_F$. GAT learns a representation z_i for each node v_i to capture the relationship between nodes. The key idea of GAT is to introduce an attention mechanism to compute the weights for

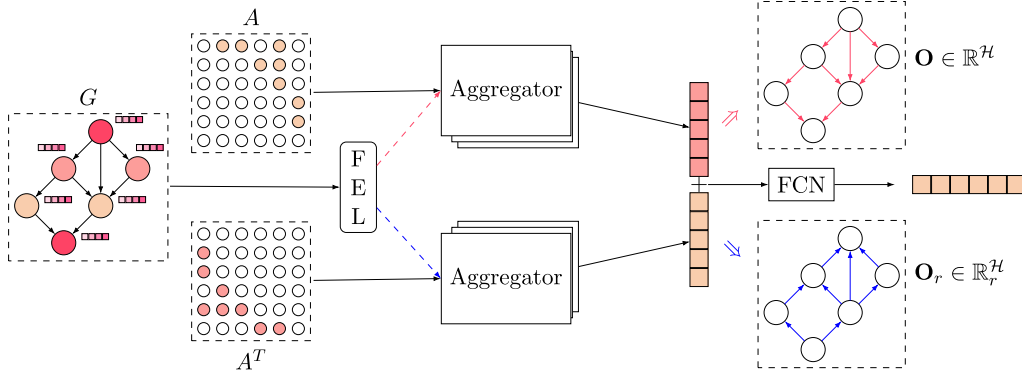


Fig. 4. Structure of BiGNN. Given a DAG G , we have its adjacency matrix A and feature list of nodes X . The Feature Extraction Layer (FEL) projects X to a desired hidden presentation. With the output of FEL, the bi-directional representations of the DAG A and A^T are fed into the Aggregator, which include GAT layers, linear projection layers, and activation. The FCN is applied to handle the output of the concatenated vector from the Aggregator and get the feature representation of G .

Table 1

Parameters of BiGNN.

Layer	Parameter number
GAT Layer 1	$Q + K: 2 \times 32 \times 2 + \text{bias}: 32 \times 2 + \text{linear}: 64 \times 32 = 2240$
GAT Layer 2	$Q + K: 32 \times 2 + \text{bias}: 32 + \text{linear}: 32 \times 32 = 1120$
GAT Layer Reversed 1	$Q + K: 2 \times 32 \times 2 + \text{bias}: 32 \times 2 + \text{linear}: 64 \times 32 = 2240$
GAT Layer Reversed 2	$Q + K: 2 \times 2 + \text{bias}: 32 + \text{linear}: 32 \times 32 = 1120$
Linear Layer	$W: 64 \times 32 + \text{bias}: 32 = 2080$
FEL	$W: 32 \times 32 \times 2 + 32 \times 64 \times 4 + \text{bias}: 32 \times 4 + 64 \times 2 = 10496$
total	19296

connections between nodes and then use these weights to perform weighted aggregation of the features of neighboring nodes. The output feature representation of v_i can be calculated by Eq. (8), where $N(i)$ represents the set of neighboring nodes of node v_i , and α_{ij} is the attention weight of node v_i to neighbor node v_j . The calculation of α_{ij} is shown in Eq. (9).

$$z_i = \text{ReLU}(\sum_{j \in N(i)} \alpha_{ij} W x_j) \quad (8)$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in N(i)} \exp(e_{ik})} \quad (9)$$

$$e_{ij} = \text{LeakyReLU}(a^T [W x_i \parallel W x_j]) \quad (10)$$

In Eq. (10), a is a learnable attention parameter vector, $\parallel$ represents vector concatenation, and W is the weight matrix. In the aggregator, several GATs are stacked to learn a better representation of the node, and in each layer, GAT receives the same node connection information and renews the node representation. Let $Z = \{z_1, z_2, \dots, z_n\}$ denote the feature set of all nodes in the entire graph, and we can calculate the output of each layer as Eq. (11), where Z^l is the output of the l th layer, $Z^0 = X_F$ when $l = 1$, and FC is a linear projection process.

$$Z^l = \text{GAT}(FC(Z^{l-1}), E) \quad (11)$$

The final graph embedding is a combination of features from both directions, including O and O_r . Their representation are shown as Eqs. (12) and (13), where L is the last layer of the stacked GAT, $+$ means the forward direction and $-$ means the backward one.

$$O = Z_+^L \quad (12)$$

$$O_r = Z_-^L \quad (13)$$

Finally, we use a Fully Connected Network (FCN) to project the graph embedding into the required feature vector. The process of generating the graph representation X_G , as well as the structural feature, is shown as Eq. (14).

$$X_G = \text{FCN}(O \parallel O_r) \quad (14)$$

In our implementation, BiGNN serves as the base model of Feature Encoder and Structure Encoder. They are components of the Network Encoder, where BiGNN is combined in multiple layers, shown in Fig. 3(a). In BiGNN, features from parent nodes and child nodes are

Table 2

Hardware factors affecting energy consumption.

Hardware	Factors
CPU	model, core number, clock frequency, cache sizes
GPU (TPU, NPU)	model, type, core number, clock frequency, memory capacity
Memory	type (e.g., DDR3, DDR4), speed, capacity
Storage	type (e.g., HDD, SSD), IO speed, capacity, rotational speed (for HDD), interface (e.g., SATA, NVMe)
Network	NIC model, speed
System	type (e.g., Android, Ubuntu), architecture (e.g., 32-bit, 64-bit)
Sensor	temperature, voltage, maximum power, fan speed

extracted separately in the way of applying the transposition of the adjacency matrix, leading to information learning in a bi-directional way.

Table 1 shows the number of parameters in BiGNN, where the forward and backward GATs are each stacked twice. Q and K are used to compute the attention coefficients, the Linear Layer maps the output of GAT to a lower dimension, and FEL stands for the fully connected network. The total number of parameters for a BiGNN module is 19,296.

4.4. Device Encoder

Apart from the neural network itself, factors that affect its inference of energy consumption include hardware features involving CPU, GPU, memory, and other related indicators. The specific potential impacts are illustrated in Table 2.

Let the hardware device parameters be denoted as a set of numbers, represented by a vector $X_H = x_1, x_2, \dots, x_n$, where x_i represents the parameter at index i . For example, x_1 could represent the CPU frequency, x_2 the GPU frequency, x_3 the memory capacity, and so on. In our method, X_H is fed into an FCN to get its feature representation c .

The network structure encoding h and device parameter encoding c will serve as inputs to the predictor through operations such as addition, multiplication, or concatenation during inference. In our implementation, we choose concatenation as the fusion method of the two vectors of encoding.

4.5. Predictor

In feature fusion, learning high-quality representations of entities is crucial, as it allows mapping different types of features into a unified space for better integration and utilization of these features [37]. This is where the role of the predictor comes into play.

The input of the predictor is concatenated from two feature vectors h and c obtained from the structure encoder and Device Encoder. After concatenation, the vector is fed into another FCN to get the energy consumption prediction e .

5. Training process

The two-stage training method trains the Network Encoder, including pre-training and fine-tuning. In this section, we provide the Graph Masking method for pre-training, several different fine-tuning methods, and designs of their objective function.

5.1. Pre-training

In the training phase of the Network Encoder, a masked autoencoder task is employed, where a random subset of node features and edges are masked. Then the masked graph is fed into the encoder to get the feature representation. During decoding, we re-mask the selected node features in the way of node category reconstruction. Finally, the decoder is applied to reconstruct the node category and the Network Encoder can generate the feature representation of the network structure. Cross-entropy loss (CELoss) is used to reconstruct the original graph. The working flow of pre-training is shown as Algorithm 1.

Graph Masking. Given the adjacency matrix A of a DAG, where A_{ij} represents whether there is a directed edge from node i to node j . A is an $n \times n$ matrix, where n is the number of nodes. Let M denote the mask matrix and the shape of M is the same as the one of A . Next, randomly select existing edges in A with a probability $p \in [0, 1]$. For each element $A_{ij} = 1$, set M_{ij} to 1 with the probability p , indicating that the edge is retained, and set M_{ij} to 0 with $1 - p$, indicating that the edge is removed. Other elements in M is set to 0. Finally, multiply A by M to obtain the masked adjacency matrix A_M . This can be achieved through matrix element-wise multiplication, where $A_{Mij} = A_{ij} \times M_{ij}$.

Loss Function. To address the objective of reconstructing the masked graph structure, we employ the CELoss. This loss function is commonly used for classification tasks, which aligns with the nature of our graph masking method. By utilizing CELoss, SDEnergy focuses on accurately reconstructing the masked feature of nodes in the graph, effectively learning the patterns and characteristics of the graph structure. This choice ensures that the model emphasizes recovering the masked node feature, contributing to the performance of the graph reconstruction task. For the node feature reconstruction task, the CELoss of Network Encoder (NE) is shown as Eq. (15).

$$Loss_{NE} = -\frac{1}{N_{\mathcal{V}_{[M]}}} \sum_{i \in \mathcal{V}_{[M]}} \sum_{c=1}^C y_{ic} \log(\hat{y}_{ic}) \quad (15)$$

$\mathcal{V}_{[M]}$ is the set of masked nodes and $N_{\mathcal{V}_{[M]}}$ represents the number of them. C is the number of candidate categories of a node and $y_{ic} = 1$ if the category of it is c , otherwise $y_{ic} = 0$. The probability of its predicted category is denoted as $\hat{y}_{ic}$.

Algorithm 1 Process of Pre-training

- 1: Randomly generate a matrix M with the same shape as A
- 2: Apply mask on A as input of *NetworkEncoder*: $A_M = A \times M$
- 3: **while** not converged **do**
- 4: Update model weights of *NetworkEncoder* by descending $Loss_{NE}$
- 5: **end while**
- 6: Derive trained *NetworkEncoder* for later fine-tuning

5.2. Fine-tuning

During the fine-tuning process, the neural network's corresponding graph structure and device parameters are used as inputs. They are encoded using two encoders to obtain feature representations. The predictor is then used to transform the outputs of the two encoders into predicted energy consumption data. The model learns to adapt to the desired task requirements by minimizing the difference between the predicted results and the collected energy consumption data by optimizing the loss function. The working flow of fine-tuning is shown as Algorithm 2.

Fine-tuning with Differential Learning Rates. We assign different learning rates to the two encoders and the predictor and train them simultaneously. By using different learning rates, we can control the weight update speed of each component during the fine-tuning process. This method allows the model to fine-tune the overall architecture while giving different attention and adjustment levels to different components. As a result, the encoders can continue learning and adapting to specific patterns and features in the fine-tuning dataset, while the predictor can focus on adjusting its ability to predict energy consumption.

Fine-tuning with Frozen Encoders. The encoders for the graph structure and device parameters are frozen, and only the energy predictor is trained. By freezing the encoders, we preserve the feature representations of the graph structure and device information learned during the pre-training phase and only adjust and optimize the energy predictor. This method allows for a more focused approach to the task of energy prediction, reducing the range of parameter updates and accelerating the convergence speed of the fine-tuning process. Additionally, freezing the encoders provides a regularization mechanism during training, helping to prevent overfitting.

Fine-tune Target. When predicting energy consumption data or other data of numeric type, the Mean Squared Error (MSE) loss function can be used. MSELoss is a common loss function for regression tasks. However, it is sensitive to outliers, so the model may overly focus on these outliers and neglect the predictions for other samples. MSELoss sums the squared differences between all predicted values and true values, without considering the sparsity of the differences. In the case of energy consumption data prediction tasks, certain energy consumption values may be zero or very close to zero. In such cases, using MSELoss may make it challenging for the model to capture this sparsity feature accurately. Due to the significant differences in the magnitude of the energy consumption data, we utilize MAPE (Mean Absolute Percentage Error) Loss to address this concern in fine-tuning (FT), which is shown as Eq. (16).

$$Loss_{FT} = \frac{1}{N_D} \sum_{i \in S_D} \frac{|e_i - \hat{e}_i|}{e_i} \quad (16)$$

N_D is the size of network-device-energy dataset S_D . The true value and predicted value of energy consumption are denoted as e_i and $\hat{e}_i$.

6. Experiment

Ablation experiments and comparative experiments are designed and conducted to validate the effectiveness, accuracy, and transferability of the proposed method. Components to be verified in the

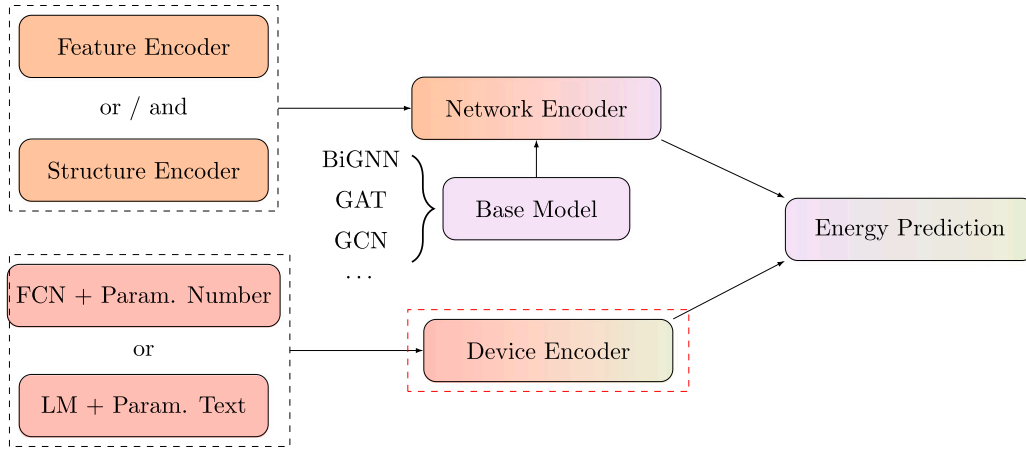


Fig. 5. Components to be Verified. The part enclosed by the red dashed line can be omitted, the Base Model is one selected from the brace, and "or/and" indicates that they can be replaced or combined. We perform experiments to (1)compare different Network Encoders, (2)compare different Device Encoders including the Language Model (LM), (3)verify the necessity of the Feature Encoder and the Structure Encoder, (4)verify the necessity of the Device Encoder. Additionally, we compare SDEnergy with other methods in the field of energy consumption prediction and compare several training methods. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Algorithm 2 Process of Fine-tuning

- 1: Given the adjacency matrix A as input of *NetworkEncoder*
- 2: Given the device parameters X_H as input of *DeviceEncoder*
- 3: **while** not converged **do**
- 4: Derive structure encoding: $h = \text{NetworkEncoder}(A)$
- 5: Derive device encoding: $c = \text{DeviceEncoder}(X_H)$
- 6: Predict energy consumption: $e = \text{predictor}(\text{concat}(h, c))$
- 7: Update model weights of *NetworkEncoder*, *DeviceEncoder* and *predictor* by descending Loss_{FT}
- 8: **end while**
- 9: Derive trained model containing above 3 parts

experiment are shown as Fig. 5, where the part enclosed by the red dashed line can be omitted, the Base Model is one selected from the brace, and "or/and" indicates that they can be replaced or combined. This section first introduces the preparatory steps for the experiments, followed by ablation experiments, comparative experiments between the encoder and other methods, accuracy comparison experiments of the overall model, and other experiments to verify the overall performance.

6.1. Experimental setup

To gather the data for training and demonstrate the transferability of SDEnergy through experiments, this section introduces the different devices on which our neural network operates and the approach used to collect energy consumption data. The collected device parameters are utilized for training the Device Encoder, while the energy consumption data is primarily employed for training the predictor during the fine-tuning phase.

Hardware. We focus on the energy consumption of neural networks on IoT devices. Therefore, Nvidia Jetson Nano b01 is selected as our device. By modifying software settings such as CPU frequency, GPU frequency, memory size, and maximum power, a large amount of data can be collected with different device configurations on the same device. The adjustable parameters for each hardware device are described in text format in Table 3. It must be pointed out that as the input of the Device Encoder, the parameters in the table are the result of querying. By modifying the relevant parameters using software tools, the corresponding description can be retrieved during the neural network inference runtime. Energy consumption data is collected for each

Table 3

Adjustable parameters of hardware device.

Description
SOC family: tegra210 Machine:NVIDIA Jetson Nano Developer Kit
Online CPUs: { <i>online_cpu_id</i> } CPU Cluster Switching: Disabled
CPU{ <i>cpu_id</i> }: Online={ <i>is_online</i> } CurrentFreq={ <i>cpu_freq</i> }
CPU{ <i>cpu_id</i> }: ...
GPU CurrentFreq={ <i>gpu_freq</i> }
Fan: speed=192
NV Power Mode: { <i>max_power</i> }
...
Adjustable Parameter
<i>online_cpu_id</i> : id list of running cpus
<i>cpu_id</i> : id of a cpu
<i>is_online</i> : 1 or 0 if this cpu is running or not
<i>cpu_freq</i> : frequency of a cpu
<i>gpu_freq</i> : frequency of a gpu
<i>max_power</i> : maximum power of this device
...

specified parameter sequence, thereby providing different parameter information for a single device and expanding the dataset.

Dataset generation. To get enough neural network structures to train our model, we apply nasbench-101 [38] to generate the dataset. Nasbench-101 is a dataset for neural architecture search (NAS), which is designed to help researchers evaluate the performance of different neural network architectures. The dataset contains a limited search space for the architectures, including neural network structures with different depths, widths, and connection modes, which can be used to build various neural network architectures.

The neural network is run on IoT devices to collect energy consumption data. For a neural network $\mathcal{N}$ and device D pair, the formula to compute the collected energy consumption data $E_{\mathcal{N}D}$ is shown as Eq. (17). We randomly collect neural network architectures from the nasbench-101 dataset, and we have access to several distinct device environments. By pairing a neural network with a device environment, we form an $\mathcal{N} - D$ pair. These energy consumption data are utilized for the training and testing of our model.

$$E_{\mathcal{N}D} = \frac{1}{k-1} \left(0 \times \sum_{i=1}^{\eta} E_i + \sum_{j=1}^{k-1} E_j \right) - E_0 \quad (17)$$

η represents the number of inference iterations performed by the neural network before the inference really runs, allowing the device power to reach a steady state. Afterward, the average value of the

Table 4

An example of the dataset format.

Hash Code	Energy consumption
0000fa05697179112aaf69c2f0a51a0f	19.054073470893844
00011986aeb27419a476d219f3d9ba14	26.399159005070224
000170f80dad8e2ef0b85c11c483eec1	20.52059274096756
...	...

inference energy consumption E_j is calculated over k inference runs by randomly initializing the parameters of the network. Finally, the silence power consumption E_0 needs to be subtracted. Without considering energy E_i of the preparation stage, the formula to compute the inference or silence energy consumption is shown as Eq. (18).

$$\sum_{j=1}^{k-1} E_j = \sum_{j=1}^{k-1} \frac{V_{j+1} + V_j}{2} \times \frac{I_{j+1} + I_j}{2} \times t_{j+1} \quad (18)$$

V_j and I_j represent the voltage and current values during the j th inference, and t_j represents the duration of it. The calculation of E_0 follows the same procedure without the inference process of the neural network, and the value of k is larger.

Based on the nasbench-101 dataset, we conduct energy consumption data collection for over 6000 random network architectures across various device environments. By adjusting the aforementioned Adjustable Parameter, the device environments used in the experiments encompass 8 distinct settings, which means that the total number of data is more than 48k. The format of the collected dataset is shown as Table 4 and the dataset is publicly available. The term Hash Code refers to the hash encoding of network architectures within the nasbench-101 dataset. Energy Consumption denotes the actual collected energy usage data, which has been subjected to Min-Max normalization processing.

Hyperparameter Settings. To achieve optimal performance, we conduct experiments with various hyperparameter configurations. Based on the computational capabilities such as memory of our training equipment, we identified a relatively optimal combination of hyperparameters. For SDEnergy, we set the batch size to 2^{10} , the learning rate during the pretraining phase to 1×10^{-4} , and the number of stackings of the base model with batch normalization to 16. The configuration of these fundamental hyperparameters ensures that the model converges rapidly during training and avoids the occurrence of overfitting. When employing NeurstrucEnergy, the number of stackings is set to 6 to achieve the best result of it. After multiple adjustments, we observe that a mask rate of 75% for G_v and 25% for G_e during pretraining led to faster convergence and better results. As for the fine-tuning phase, the learning rate for the Device Encoder and the Predictor of the model is set to 1×10^{-3} , and the one of the Network Encoder to 1×10^{-5} . This hyperparameter configuration ensures the effectiveness of the training process and achieves lower MAPE within 600 training epochs. Additionally, we also attempt to apply language model (LM) as the Device Encoder to handle the pure text description of device parameters, namely SDEnergy-LM, and its learning rate is also set to 1×10^{-5} . Considering the scale of the Language Model (LM) and the capabilities of our training equipment, the batch size for SDEnergy-LM is relatively smaller than SDEnergy. This adjustment is often necessary to accommodate the increased computational demands of larger models while ensuring that the training process remains feasible within the constraints of the available hardware. The detail of hyperparameters is shown in Table 5.

6.2. Network Encoder comparison

In order to learn the representation of graph-structured data and extract features from neural network graph structures, we employ Graph Neural Networks (GNN). The dataset used for training and testing is derived from nasbench-101, which stores a large number of graph structures. Each graph structure in the dataset is available to

Table 5

Hyperparameter settings.

Model	Hyperparameters
SDEnergy	batch size of pretrain = batch size of fine-tune = 1024, $H = H' = 32$, learning rate of pretrain = 1×10^{-4} , number of stacked layers = 16, mask rate of $G_v = 75\%$, mask rate of $G_e = 25\%$, activation function = ReLU, pretrain epochs = 1000, fine-tune epochs = 600, criterion = MAPE, learning rate of the Device Encoder and the Predictor = 1×10^{-3} , learning rate of the Network Encoder = 1×10^{-4} , and optimizer=AdamW.
SDEnergy-LM	batch size of LM training = 8, batch size of fine-tune = 128, learning rate of the Device Encoder = 1×10^{-5} , and others are the same as SDEnergy.

Table 6

Network encoder comparison (Without Device Encoder).

GNN	MAPE-train	MAPE-test	Pretrain loss CELoss
SAGEConv	6.0538%	7.5699%	$< 1e^{-5}$
GCNConv	9.2217%	10.3605%	0.2692
SGConv	8.0614%	9.3166%	0.2489
GATConv	7.0948%	8.2088%	$3.6 \times 1e^{-5}$
BiGNN (ours)	3.8085%	5.3467%	$< 1e^{-5}$

know and contains information about node types and edge connections. In the phase of graph representation learning, we select several GNN models, such as Graph Convolutional Networks (GCN), Graph Attention Networks (GAT), and GraphSAGE, as comparative models for our experiments. These models are the base models for our unsupervised learning with deep neural networks. The data augmentation method used in unsupervised learning is random masking, where for each graph structure in the training set, a subset of nodes and edges are randomly selected and masked, removing their corresponding features and connection information. The masked graph structures are then used as input, and the selected GNN deep model is trained with the objective of predicting the categories of the masked nodes.

During the training process, Cross-Entropy Loss is employed as the loss function to measure the discrepancy between the predicted results and the true classes. The loss function is applied to node feature recovery. Finally, the performance of the trained model is evaluated on the test set, where the loss value can be used as an evaluation metric to measure the accuracy of node feature and edge information recovery. A lower loss value indicates that the model can recover node category information more accurately. Based on the experimental results, we can determine which GNN model performs better in the task of graph-structured data representation learning. Besides, the Network Encoder Comparison here is carried on with FCN as the Device Encoder and the result is shown in Table 6.

As BiGNN applies bi-directional GNN, it can capture more network structure information. The base part of BiGNN is GATConv, but after two layers of stacking and bi-directional splicing, it forms a relatively complex structure, which is more efficient in feature encoding.

6.3. Device Encoder comparison

Apart from applying FCN as the Device Encoder on device parameters of numeric type in SDEnergy, we utilize language models for representation learning of device description information, i.e., textual data, to extract device features. Since assessing the learning capability of the language model or FCN alone is not significantly important for the task at hand, we combine the BiGNN with strong graph-structured data learning abilities with the language models for comparative experiments. In the experiments, the same GNN is employed, and different pre-trained language models, such as BERT and GPT, are chosen as comparative models.

Table 7

Device encoder comparison (With BiGNN).

Method	MAPE-train	MAPE-test
BERT	8.7182%	9.4896%
GPT2	6.8836%	7.3888%
CLIP Text Encoder	12.9116%	15.9132%
FCN (ours)	3.8085%	5.3467%

Table 8

Necessity of feature encoder or structure encoder (With BiGNN).

Method	MAPE-train	MAPE-test
Only Feature Encoder	4.1214%	5.6717%
Only Structure Encoder	3.8976%	5.4213%
Feature + Structure (ours)	3.8085%	5.3467%

Table 9

Necessity of device encoder (With BiGNN).

Method	MAPE-train	MAPE-test
Without Device Encoder	12.1035%	15.0432%
With Device Encoder (ours)	3.8085%	5.3467%

Initially, unsupervised learning is employed to fine-tune the language model and obtain text features. Then, supervised learning is conducted using energy consumption data to train a comprehensive model that incorporates both the Network Encoder and the Device Encoder. **MAPE Loss** is used as the loss function and evaluation metric. The result is shown in Table 7.

Language models in Table 7 are pre-trained and FCN is only trained in the fine-tuning phase. As shown in the table, FCN achieves the best result but data preprocessing is needed to apply it. Moreover, it requires manual extraction of useful digital features. LM is a ready-to-use method but its training time is relatively longer and the accuracy is a little worse.

6.4. Ablation study

We conduct an ablation study to validate the necessity of each component in the proposed method. The following components are involved:

- **Necessity of Feature Encoder and Structure Encoder in the Network Encoder.** This analysis includes examining the necessity of feature masks and edge masks as well. For example, we remove the structure encoder and utilize unmasked edges as one of the inputs to the feature encoder. We observe the recovery of features after masking and monitor changes in prediction results. The result is shown in Table 8.
- **Necessity of Device Encoder.** In this case, we exclude the Device Encoder and directly employ the output of the Network Encoder as the input to the predictor during the fine-tuning process. We observe the variations in the accuracy of energy consumption prediction. The result is shown in Table 9.

By comparing the loss results from pre-training and the accuracy of energy consumption prediction, we can assess the necessity of these components. If removing any component leads to a significant decrease in model performance, it indicates that the component is necessary for graph-structured data representation learning or energy consumption prediction.

As shown in Table 8, the Structure Encoder seems to be a more useful part, and the model with both feature and structure encoders achieves a little better results. However, as the framework of the two encoders is exactly the same and only the masked part is different, they only change the complexity of the model and the length of the output feature. In Table 9, the model with the Device Encoder achieves a much

Table 10

Prediction accuracy comparison.

Model	MAPE-train	MAPE-test
NeuralPower [10]	24.2005%	24.3905%
PreVIous [12]	24.2005%	24.3905%
PMLP [39]	17.2404%	17.8116%
RECT-L [40]	12.9543%	13.8899%
NeurstrucEnergy [41]	13.1140%	15.4343%
NeurstrucEnergy+Device Encoder	5.4862%	6.8016%
SDEnergy (ours)	3.8085%	5.3467%
SDEnergy-LM	6.8836%	7.3888%

better result. The model without the Device Encoder may encounter situations where the same network corresponds to different energy consumption data, which is also one of the main issues to be addressed in SDEnergy.

The complexity of the model is enhanced by integrating two types of inputs, including graph structural information and device encoding information, stacking modules in the model, and fusing features from multiple modalities. Compared to using GNN solely to process graph-structured data or employing Device Encoder to handle device information, SDEnergy provides a more comprehensive consideration.

6.5. Prediction accuracy comparison

We proceed with a comparative experiment to assess the overall accuracy of energy consumption prediction using the selected superior Graph Neural Network (GNN) and Language Model (LM) as components of the integrated model. Although the LM accuracy is slightly lower than FCN in SDEnergy, it is more convenient because it omits the steps of digitizing device parameters and directly takes the text description as input. We use multiple energy consumption data collected from multiple equipment parameters as the dataset of our experiment. Each network structure in the dataset corresponds to multiple energy consumption data because the devices are different. The result is shown in Table 10, and SDEnergy-LM is where LM replaces FCN as the Device Encoder to deal with device description instead of numeric parameters.

The advantage of SDEnergy lies in its use of bidirectional graph neural network learning to learn representations of graph structures, and it incorporates considerations for the fusion of device information, thereby achieving state-of-the-art performance. Other works have overlooked the impact of device encoding on prediction outcomes. NeuralPower and PreVIous are deficient in representation learning for graph-structured data and require manual feature extraction. Although PMLP and RECT-L can relatively well learn graph structure features, they neglect the influence of child nodes on parent nodes. NeurstrucEnergy employs bidirectional graph neural networks for graph structure representation, and although its performance improves with the addition of device encoding, the primary factor affecting its performance is the lack of a Graph Auto Encoder training approach.

The evaluation metric for the experiment is the MAPE of the energy consumption data prediction. As shown in Table 10, SDEnergy can accurately predict the energy consumption of the networks under different equipment parameters, and the MAPE value of the test dataset is below 6%, which achieves state-of-the-art. Besides, as NeurstrucEnergy is applied in a similar situation to predict the energy consumption of neural networks, we also combine it with our Device Encoder to conduct an experiment. As shown in the table, a simple additional encoder can achieve better results. However, the results of SDEnergy-LM are a little worse, and according to the observation in the experiment, the fine-tuning time of SDEnergy-LM is 5 times longer than SDEnergy, which is obvious. Therefore, to get more accurate results, SDEnergy should be applied to digital data, and if text information is used directly without data preprocessing, SDEnergy-LM is an alternative option.

Besides, at the training phase, several approaches are employed, and the result is shown in Table 11. The approaches include pre-training-based, encoder freezing, and end-to-end training:

Table 11
Training method comparison (With BiGNN).

Method	MAPE-train	MAPE-test
Network Encoder Freezing	14.5673%	15.2959%
End-to-End	6.0011%	6.8872%
Pretrain-Finetune (ours)	3.8085%	5.3467%

- **Pre-training-based.** Two encoders and a predictor are used, with different learning rates assigned to the encoders. This facilitates fine-tuning of the encoders and training of the predictor.
- **Encoder Freezing.** One or both of the encoders are frozen, keeping their parameters unchanged, while only updating the parameters of the predictor.
- **End-to-End Training.** In this experiment, the neural network graph structure data and device textual information are directly used as inputs to predict energy consumption data.

In Table 11, the pre-training-based method achieves the best result, which is because of the pre-learning of the network structure. However, the pre-trained Network Encoder cannot be used directly without fine-tuning, whose result is much worse and is shown in the Network Encoder freezing row. In addition, an end-to-end experiment is conducted and its convergence speed is relatively slow.

6.6. Discussion

In the comparison of Network Encoders, our proposed BiGNN yields the best performance, as it can gather bi-directional feature information from both parent and child nodes, considering the entire graph structure. In contrast, in the comparison of Device Encoders, we explore the application of a language model to device feature encoding, allowing direct transformation of device textual descriptions into features. However, the language model exhibits lower accuracy compared to FCN and significantly slower processing speeds. On the other hand, FCN requires manual extraction of numeric device parameters, which can be cumbersome.

In addition to the choice of the Network Encoder and the Device Encoder, we further verify the necessity of Feature Encoder and Structure Encoder within the Network Encoder. Our results indicate that both these components contribute to the encoding of graph structures. The importance of Device Encoder is also worth investigating, demonstrating SDEnergy's transferability across various device conditions. The outcomes reveal that the Device Encoder effectively enhances the capabilities of the Network Encoder's single branch.

Building upon the aforementioned experiments, we select the proposed BiGNN as the base model for the Network Encoder and employ FCN as the Device Encoder. We compare this approach with previous methods, demonstrating that SDEnergy achieves state-of-the-art results. To ensure fairness in the experiments, we incorporate our Device Encoder into NeurstrucEnergy, although some performance gaps are observed.

Furthermore, we conduct experimental trials with SDEnergy-LM, which, although not meeting our expectations, eliminates the need for manual collection of device parameters. It facilitates automation, particularly in scenarios with a diverse range of device types, as it only requires device description text.

Lastly, we validate the effectiveness of our two-stage training approach, where the unsupervised learning of the Network Encoder enhances its capacity to encode graph-structured data.

7. Conclusion

In this work, we introduce SDEnergy, a method for efficiently predicting neural network energy consumption without device-specific inference. Our approach encompasses a Network Encoder designed to

process the features and structural information across various layers of neural networks, as well as a Device Encoder that handles device encoding information. Furthermore, we have enhanced the processing of device description information using some NLP models. Our training process incorporates pre-training, fine-tuning, and self-supervised learning with a masking strategy. Experiments show that our BiGNN and FCN-based encoders excel in energy prediction, with ablation studies underscoring the value of both encoders. SDEnergy achieves state-of-the-art performance in energy consumption prediction. Future plans include expanding our dataset and exploring additional network architectures to enhance our predictive models.

Abbreviation table

A	adjacency matrix of $\mathcal{G}$
A_{ij}	element of i th row and j th column in A
α_{ij}	attention weight of node v_i to neighbor node v_j
c	feature vector of x
cls_v	class of v
D_{feat}	feature dimension of x_v
E	node collection of $\mathcal{G}$
E	a subset of E , where edges is masked
e	energy consumption
e_{ij}	edge connecting v_i and v_j
e_[M]	masked edges
G	a directed acyclic graph
G	a graph-structure dataset
$\mathcal{G}$	a graph in G
$\mathcal{G}_e$	same as $\mathcal{G}$, but edges are randomly masked
$\mathcal{G}_v$	same as $\mathcal{G}$, but nodes are randomly masked
G_e	edges of graph G
G_v	nodes of graph G
H	dimension of O
h	structure encoding of a neural network
I_j	current value during the j th inference
N_{edge}	number of edges in graph G
N_{node}	number of nodes in graph G
N(i)	set of neighboring nodes of node v_i
O	hidden presentation of a structure
t_j	time of the j th inference
V	edge collection of $\mathcal{G}$
$\bar{V}$	a subset of V where nodes is masked
V_j	voltage value during the j th inference
v	a node of graph G , also a layer of a neural network
v_i	the i th element of node list
v_[M]	masked nodes
X	feature representation of all nodes in a DAG
X_F	hidden representation output of FEL
x	device parameters
x_{sth}	device parameter of a hardware named sth
x_v	feature embedding of node v
y	label of node category
Z^l	output of the l th layer
Z₊^L	output of the last layer in the forward direction
Z₋^L	output of the last layer in the backward direction
θ	weights of structure encoder
η	number of inference iterations performed by the neural network

CRedit authorship contribution statement

Chaopeng Guo: Writing – review & editing, Project administration, Methodology, Investigation, Formal analysis, Conceptualization. **Shiyu Wang:** Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Investigation. **Ruolan Xie:** Writing – review & editing, Software, Investigation. **Jie Song:** Writing – review & editing, Supervision, Formal analysis.

Funding

This paper is supported by the National Natural Science Foundation of China (Grant No. 62302086), the Natural Science Foundation of Liaoning Province, China (Grant No. 2023-MSBA-070) and the Fundamental Research Funds for the Central Universities, China (Grant No. N2317005).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] J. Barr, Amazon Ec2 Update—Inf1 instances with aws inferentia chips for high performance costeffective inferencing, 2019, URL <https://shorturl.at/DEhyU>.
- [2] K. Freund, Google cloud doubles down on nvidia gpus for inference, 2019, URL <https://www.forbes.com/sites/moorinsights/2019/05/09/google-cloud-doubles-down-on-nvidia-gpus-for-inference>.
- [3] D. Hernandez, T.B. Brown, Measuring the algorithmic efficiency of neural networks, 2020, arXiv:2005.04305.
- [4] L. Yin, M. Zhuang, J. Jia, H. Wang, Energy saving in flow-shop scheduling management: An improved multiobjective model based on grey wolf optimization algorithm, in: W. Xiao (Ed.), Math. Probl. Eng. (2020) 1–14, <https://doi.org/10.1155/2020/9462048>, URL <https://www.hindawi.com/journals/mpe/2020/9462048/>.
- [5] Y. Yang, Z. Zhang, Y. Zhou, C. Wang, H. Zhu, Design of a simultaneous information and power transfer system based on a modulating feature of magnetron, IEEE Trans. Microw. Theory Tech. 71 (2) (2023) 907–915, <https://doi.org/10.1109/TMTT.2022.3205612>, URL <https://ieeexplore.ieee.org/document/9900115/>.
- [6] J. Frankle, G.K. Dziugaite, D.M. Roy, M. Carbin, Pruning neural networks at initialization: Why are we missing the mark? in: International Conference on Learning Representations, OpenReview.net, Online, 2021, pp. 1–41.
- [7] S. Anwar, K. Hwang, W. Sung, Structured pruning of deep convolutional neural networks, ACM J. Emerg. Technol. Comput. Syst. 13 (3) (2017) 1–18, <https://doi.org/10.1145/3005348>.
- [8] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, D. Kalenichenko, Quantization and training of neural networks for efficient integer-arithmetic-only inference, in: IEEE Conference on Computer Vision and Pattern Recognition, IEEE, Salt Lake City, State of Utah, the United States, 2018, pp. 2704–2713, <https://doi.org/10.1109/CVPR.2018.00286>.
- [9] S. Han, H. Mao, W.J. Dally, Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding, 2016, arXiv:1510.00149.
- [10] E. Cai, D.C. Juan, D. Stamoulis, D. Marculescu, NeuralPower: Predict and deploy energy-efficient convolutional neural networks, 2017, arXiv:1710.05420.
- [11] X. Dai, Y. Jia, P. Vajda, M. Uyttendaele, N.K. Jha, P. Zhang, B. Wu, H. Yin, F. Sun, Y. Wang, M. Dukhan, Y. Hu, Y. Wu, ChamNet: Towards efficient network design through platform-aware model adaptation, in: IEEE Conference on Computer Vision and Pattern Recognition, IEEE, Long Beach, California, the United States, 2019, pp. 11390–11399, <https://doi.org/10.1109/CVPR.2019.01166>.
- [12] D. Velasco-Montero, J. Fernandez-Berni, R. Carmona-Galan, A. Rodriguez-Vazquez, PreVlous: A methodology for prediction of visual inference performance on IoT devices, IEEE Internet Things J. 7 (10) (2020) 9227–9240, <https://doi.org/10.1109/JIOT.2020.2981684>.
- [13] C.F. Rodrigues, G. Riley, M. Luján, SyNERGY: An energy measurement and prediction framework for convolutional neural networks on jetson TX1, in: International Conference on Parallel and Distributed Processing Techniques and Applications, CSREA Press, Las Vegas, Nevada, the United States, 2018, pp. 375–382.
- [14] D. Justus, J. Brennan, S. Bonner, A.S. McGough, Predicting the computational cost of deep learning models, in: IEEE Conference on Big Data, IEEE, Seattle, United States, 2018, pp. 3873–3882, <https://doi.org/10.1109/BigData.2018.8622396>.
- [15] C. Yao, W. Liu, Z. Liu, L. Yan, S. Hu, W. Tang, EALI: Energy-aware layer-level scheduling for convolutional neural network inference services on GPUs, Neurocomputing 507 (2022) 265–281, <https://doi.org/10.1016/j.neucom.2022.08.025>.
- [16] J. Getzner, B. Charpentier, S. Günnemann, Accuracy is not the only metric that matters: Estimating the energy consumption of deep learning models, 2023, arXiv:2304.00897.
- [17] T.J. Yang, Y.H. Chen, J. Emer, V. Sze, A method to estimate the energy consumption of deep neural networks, in: Asilomar Conference on Signals, Systems, and Computers, IEEE, Pacific Grove, California, the United States, 2017, pp. 1916–1920, <https://doi.org/10.1109/ACSSC.2017.8335698>.
- [18] E. García-Martín, C.F. Rodrigues, G. Riley, H. Grah, Estimation of energy consumption in machine learning, J. Parallel Distrib. Comput. 134 (2019) 75–88, <https://doi.org/10.1016/j.jpdc.2019.07.007>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0743731518308773>.
- [19] S. Chen, W. Guo, Auto-encoders in deep learning—A review with new perspectives, Mathematics 11 (8) (2023) 1777, <https://doi.org/10.3390/math11081777>, URL <https://www.mdpi.com/2227-7390/11/8/1777>.
- [20] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: International Conference on Learning Representations, OpenReview.net, Toulon, France, 2017, pp. 1–14.
- [21] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: International Conference on Learning Representations, OpenReview.net, Vancouver, British Columbia, Canada, 2018, pp. 1–12.
- [22] K. Xu, W. Hu, J. Leskovec, S. Jegelka, How powerful are graph neural networks? in: International Conference on Learning Representations, OpenReview.net, New Orleans, Louisiana, the United States, 2019, pp. 1–17.
- [23] F. Bi, X. Luo, B. Shen, H. Dong, Z. Wang, Proximal alternating-direction-method-of-multipliers-incorporated nonnegative latent factor analysis, IEEE/CAA J. Autom. Sin. 10 (6) (2023) 1388–1406, <https://doi.org/10.1109/JAS.2023.123474>, URL <https://ieeexplore.ieee.org/document/10084419/>.
- [24] Z. Liu, X. Luo, M. Zhou, Symmetry and graph bi-regularized non-negative matrix factorization for precise community detection, IEEE Trans. Autom. Sci. Eng. 21 (2) (2024) 1406–1420, <https://doi.org/10.1109/TASE.2023.3240335>, URL <https://ieeexplore.ieee.org/document/10072010/>.
- [25] M. Liu, X. Li, J. Li, Y. Liu, B. Zhou, J. Bao, A knowledge graph-based data representation approach for IIoT-enabled cognitive manufacturing, Adv. Eng. Inform. 51 (2022) 101515, <https://doi.org/10.1016/j.aei.2021.101515>, URL <https://linkinghub.elsevier.com/retrieve/pii/S1474034621002639>.
- [26] K. Wang, J. An, M. Zhou, Z. Shi, Q. Shi, Q. Kang, Minority-weighted graph neural network for imbalanced node classification in social networks of internet of people, IEEE Internet Things J. 10 (1) (2023) 330–340, <https://doi.org/10.1109/JIOT.2022.3200964>, URL <https://ieeexplore.ieee.org/document/9875203/>.
- [27] J. Devlin, M.W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in: Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Association for Computational Linguistics, Minneapolis, Minnesota, the United States, 2019, pp. 4171–4186, <https://doi.org/10.18653/v1/n19-1423>, arXiv:1810.04805.
- [28] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, R. Girshick, Masked autoencoders are scalable vision learners, in: IEEE Conference on Computer Vision and Pattern Recognition, IEEE, New Orleans, Louisiana, the United States, 2022, pp. 15979–15988, <https://doi.org/10.1109/CVPR52688.2022.01553>, arXiv:2111.06377.
- [29] J. He, Z. Han, Advanced sensors towards ubiquitous power internet of things, CSEE J. Power Energy Syst. 10 (3) (2024).
- [30] Z. Han, F. Xue, J. Hu, J. He, Micro electric field sensors: Principles and applications, IEEE Ind. Electron. Mag. 15 (4) (2021) 35–42, <https://doi.org/10.1109/MIE.2020.3046226>.
- [31] Y. Ouyang, J. He, J. Hu, G. Zhao, Z. Wang, S.X. Wang, Contactless current sensors based on magnetic tunnel junction for smart grid applications, IEEE Trans. Magn. 51 (11) (2015) 1–4, <https://doi.org/10.1109/TMAG.2015.2446332>.
- [32] H. Qi, E.R. Sparks, A. Talwalkar, Paleo: A performance model for deep neural networks, in: International Conference on Learning Representations, OpenReview.net, Toulon, France, 2017, pp. 1–10.
- [33] T.N. Kipf, M. Welling, Variational graph auto-encoders, 2016, arXiv:1611.07308.
- [34] Z. Hou, X. Liu, Y. Cen, Y. Dong, H. Yang, C. Wang, J. Tang, GraphMAE: Self-supervised masked graph autoencoders, in: Conference on Knowledge Discovery and Data Mining, KDD '22, Association for Computing Machinery, Washington, D.C., the United States, 2022, pp. 594–604, <https://doi.org/10.1145/3534678.3539321>.
- [35] J. Yang, F. Yang, Y. Zhou, D. Wang, R. Li, G. Wang, W. Chen, A data-driven structural damage detection framework based on parallel convolutional neural network and bidirectional gated recurrent unit, Inform. Sci. 566 (2021) 103–117, <https://doi.org/10.1016/j.ins.2021.02.064>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0020025521002061>.
- [36] J. Yang, L. Zhang, C. Chen, Y. Li, R. Li, G. Wang, S. Jiang, Z. Zeng, A hierarchical deep convolutional neural network and gated recurrent unit framework for structural damage detection, Inform. Sci. 540 (2020) 117–130, <https://doi.org/10.1016/j.ins.2020.05.090>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0020025520305107>.
- [37] Y. Xu, E. Wang, Y. Yang, Y. Chang, A unified collaborative representation learning for neural-network based recommender systems, 2022, arXiv:2205.09670 [cs], URL <http://arxiv.org/abs/2205.09670>.

- [38] C. Ying, A. Klein, E. Real, E. Christiansen, K. Murphy, F. Hutter, NAS-Bench-101: Towards reproducible neural architecture search, in: *Proceedings of the 36th International Conference on Machine Learning*, PMLR, Long Beach, California, the United States, 2019, pp. 7105–7114.
- [39] C. Yang, Q. Wu, J. Wang, J. Yan, Graph neural networks are inherently good generalizers: Insights by bridging GNNs and MLPs, 2023, [arXiv:2212.09034](https://arxiv.org/abs/2212.09034) [cs], URL <http://arxiv.org/abs/2212.09034>.
- [40] Z. Wang, X. Ye, C. Wang, J. Cui, P.S. Yu, Network embedding with completely-imbalanced labels, *IEEE Trans. Knowl. Data Eng.* 33 (11) (2021) 3634–3647, <http://dx.doi.org/10.1109/TKDE.2020.2971490>, [arXiv:2007.03545](https://arxiv.org/abs/2007.03545) [cs, stat], URL <http://arxiv.org/abs/2007.03545>.
- [41] C. Guo, Z. Zhong, Z. Zhang, J. Song, NeurstrucEnergy: A bi-directional GNN model for energy prediction of neural networks in IoT, *Digit. Commun. Netw.* (2022) 1–16, <http://dx.doi.org/10.1016/j.dean.2022.09.006>, URL <https://linkinghub.elsevier.com/retrieve/pii/S235286482200181X>.